
Verification Specification

Programmable Interrupt Controller with Advanced Scheduling

Verification environment, methodology,
verification plan and measured results

Document	Verification Specification — Programmable Interrupt Controller
Document type	Verification specification
Subject	Programmable interrupt controller, sixteen sources, hardware nesting
Revision	1.1
Date	15 September 2026
Author	Bunea Cosmin-Andrei
Status	Released for review

Contents

1	Introduction	2
1.1	Purpose of the document	2
1.2	Validity of the document	2
1.3	Definitions of terms and abbreviations	2
1.4	Requirements and implementation decisions	2
1.5	Relationship with other documents	3
1.6	Notes on the notation used	3
2	Verification methodology	3
2.1	Overview	3
2.2	Implementation strategy	4
2.3	Verification plans	5
2.4	Reference model	6
3	Verification environment	7
3.1	Overview	7
3.2	Control and drivers	7
3.2.1	Clock and reset generation	7
3.2.2	Source line stimulus	8
3.2.3	Processor handshake stimulus	8
3.2.4	Register access stimulus	8
3.3	Monitors and checkers	8
3.3.1	Bus protocol monitor	8
3.3.2	Assertion layer	9
3.3.3	Self-checking comparison	9
3.4	Simulation environment setup	10
3.4.1	Structure	10
3.4.2	Scripts	10
3.4.3	Setup and configuration	10
4	Annex	10
4.1	Verification plan	10
4.2	Measured results	17

1 Introduction

1.1 Purpose of the document

This document specifies the verification environment applied to the programmable interrupt controller, the methodology behind it, and what that environment establishes about the design.

The verification plan in the annex lists what is checked, and where each check is applied.

1.2 Validity of the document

The document is valid for the interrupt controller as currently implemented and for the verification environment as it currently exists: six block-level tests, two system-level environments, one passive bus protocol monitor and one assertion layer.

1.3 Definitions of terms and abbreviations

Term	Meaning
Assertion	A property evaluated on every clock cycle of a run, which fails the run if it is ever violated.
AXI4-Lite	Advanced eXtensible Interface, revision 4, Lite subset. The bus protocol of the controller's register port.
Block level	A test that instantiates the controller on its own and drives its ports directly.
Check	One comparison of an observed value against an expected value, producing one verdict.
Claim	The pulse with which a processor announces that it has entered the handler for the offered source.
Directed test	A test whose stimulus sequence is written explicitly rather than generated.
End of interrupt	The pulse with which a processor announces that one handler is returning.
Monitor	A passive observer that watches an interface and reports protocol violations without driving anything.
Offer	The request line and source number the controller presents.
Regression	The full set of runs, executed together and judged as a whole.
Run	One execution of one test in one configuration.
System level	A test in which the controller is integrated with a real processor and real memories and driven by a program.

1.4 Requirements and implementation decisions

The original Siemens briefs are kept in `cpu/docs/brief/`: *RV32I 3-Stage Pipeline CPU* and *Programmable Interrupt Controller with Advanced Scheduling*. They define the project scope. Instruction and CSR semantics are checked against the cited RISC-V manuals, and bus handshakes against the AXI specification. These authored specifications describe the implementation; they are not independent evidence that the implementation is correct.

Details absent from the briefs, including the register map, bus error policy and the sixteen-source claim/EOI interface, are project decisions. The CPU brief's eight-source interface and the PIC brief's sixteen-source interface still require a common interface agreement; the implemented contract is stated explicitly in the design specifications.

1.5 Relationship with other documents

Document	Relationship
Design Specification — Programmable Interrupt Controller	Specifies the design verified here. Register names, field definitions, interface behaviour and the stated limitations are taken from it, and the plan items are written in its terminology.
Design Specification — RV32I Processor Core	Specifies the processor used as the surrounding design in the system-level test, including the conditions under which it accepts an offer and emits the claim and end-of-interrupt pulses.
Verification Specification — RV32I Processor Core	Specifies the same system-level environment from the processor’s side. The system-level runs are shared between the two documents; the checks counted here are the ones that bear on the controller.
AMBA AXI and ACE Protocol Specification, ARM IHI 0022E	The rule set the protocol monitor and the protocol assertions evaluate.

1.6 Notes on the notation used

- Fixed-width type denotes a signal, a register or a field, for example `INT_ENABLE` or `AWVALID`.
- Italic type marks emphasis, or a term where a sentence defines it. Bold type marks a run-in heading.
- Verification plan items are identified as `VP-nn` and are listed in the annex.
- “The controller” is the design under verification. “The processor” is the core used as the surrounding design at system level.
- A *check* is one comparison producing one verdict. Counts in this document are counts of verdicts, not of test cases.

2 Verification methodology

2.1 Overview

Three mechanisms are used together, because each of them sees something the others cannot.

Table 2: Verification mechanisms and their reach

Mechanism	What it establishes, and what it cannot
Directed self-checking tests	Establish that a specific scenario produces a specific result. Each test carries its own expected values and prints a verdict, and the runner requires explicit completion with zero mismatches. They see only the scenarios someone thought to write.
Protocol monitor	Establishes that the register port obeys the bus protocol, continuously and independently of what the design and the driving master believe is happening. It sees protocol violations, not wrong register values.
Assertions	Establish invariants that must hold in every cycle of every run in which they are active, including cycles no test was written for. They see rule violations, not wrong values.

There is one further mechanism in the surrounding environment. A functional coverage model bound into the processor records, among other things, how many interrupts were claimed and which sources they came from. It observes the controller only indirectly, at the processor’s boundary, and it is evaluated only by the secondary simulator flow described in Section 3.3.2. On the run measured here every one of its per-source claim bins was reached except the source deliberately left masked for the whole run.

2.2 Implementation strategy

Verification is applied at system and block level.

Block level. The controller is instantiated on its own and driven directly on its source lines, its processor-facing pins and its register port. This is where the majority of the evidence comes from, for two reasons. Most of the controller’s behaviour is only reachable by a stimulus a processor would never generate — a source that withdraws in the one cycle between the offer and the claim, a rising edge that coincides exactly with a claim, a nesting limit lowered below the current depth — and most of it is only observable by reading registers at a moment a running program could not choose.

Six block-level tests are used, each with a distinct purpose rather than a distinct feature list.

Table 3: Block-level tests

Test	Purpose
Independent priority reference	Compares offers and pending masks across 275 generated cases, including all band urgency encodings and sixteen simultaneous contenders.
Feature test	Walks each capability of the controller once along its natural path: a plain level interrupt, the three levels of the priority comparison, preemption and unwinding, the depth limit, spurious detection, escalation in both modes, software triggers, edge sources, and the bus response codes.
Scheduling test	The cases where two things happen in the same cycle, or where a programmable field is set to something other than its default. These are separated from the feature test because a test that only walks the natural path cannot reach them.
Reset test	The reset contract, checked from the first cycle. A test that spends its first hundred cycles configuring the block has already destroyed the evidence a reset defect would leave.
Access-class test	That every register the map calls read-only is read-only, that every bit it calls reserved is hardwired to zero, and that the write-one-to-clear and write-any-read-legal registers behave as their access class requires.
Status test	That the hardware puts the right thing into the per-source status register, field by field and in both directions.

System level. The controller is integrated with a real processor and real memory models, and the interrupts are produced and consumed by the real handshake rather than by a driver. This level exists to establish the things the block level cannot: that a real processor’s claim timing satisfies the interface contract, that the two independent masks interact as intended, and that the register interface is usable from a program over real bus transactions.

Two environments do this. The processor block’s own system test drives the controller from a program alongside everything else the processor does. The assembled system adds runs whose subject is the controller itself: a program that raises each of the sixteen slots in turn, one that has a handler preempted from inside itself, and one that lets a source miss its deadline and then checks which handler the processor enters first. Those three reach behaviour a peripheral-driven run cannot, because the software trigger channels let a program place a request on any slot at a moment of its choosing, including slots no device is wired to.

The counted checks are in every case the subset that bears on the controller.

Bus timing variation. The processor system test is repeated under four memory timing configurations: nominal, high fixed latency, and two seeds of randomised ready-signal backpressure. The interrupt path is sensitive to this because the processor defers a claim while a data transaction is in flight, so the interval between the offer and the claim changes with the memory timing.

Pass criteria. Every comparison and protocol monitor must report zero errors. Applicable assertions must pass in Verilator. Each test completes through `finish_test`; the ModelSim runner requires `test_done=1`, zero errors and the requested parameter values. Timeouts, missing completion and simulator errors return nonzero status. The GUI also checks exact completion-banner counts and the final regression marker.

2.3 Verification plans

The verification plan is organised by design area rather than by test, so that a plan item can be traced to the behaviour it covers regardless of which test happens to implement it. Each item

names the behaviour, how it is stimulated, how the expected result is determined, and where the check is applied. The items are listed in the annex.

Table 4: Verification plan areas

Area	Covers
Source handling	Trigger types, the edge latch, the per-source enable and the software trigger channel.
Priority and scheduling	The three levels of the priority comparison, band reordering, and the processor's mask inside the resolution.
Nesting and preemption	The nesting stack, the preemption threshold, unwinding, and the depth limit.
Deadline escalation	The elapsed-time counter, both escalation modes, and the effect of an escalation on the offer.
Spurious claims	Detection, the response to it, and the accounting that keeps claims and returns balanced.
Register interface	Access classes, reserved bits, error responses and bus protocol compliance.
Reset	Reset values, reset behaviour at the interfaces, and the asynchronous release.
System integration	The handshake against a real processor, the interaction of the two masks, and use of the register interface from software.
Continuous invariants	The properties the assertion layer evaluates in every cycle of every run in which it is active.

2.4 Reference model

`pic_reference.py` independently orders eligible sources by band urgency, intra-band priority and lowest source index, using a Python sort rather than the RTL priority-key expression. With seed 715 it generates 275 cases: all 256 band-configuration encodings, sixteen forced winners, a full tie, an empty mask and an empty request set. Configurations include tied urgency values and sixteen simultaneous contenders.

`pic_tb_reference` programs each case through AXI4-Lite, then checks the offer, vector and pending mask against the generated expectations. The reference covers priority selection and masking; the directed tests below cover the sequential claim, nesting and deadline behaviour.

Other expected results are determined in one of three ways, depending on the check.

- **Precomputed constants.** Reset values, field masks, response codes and register contents after a known sequence are derived by hand from the specification and embedded in the test. This is the majority of the checks.
- **The predicted offer.** For a scheduling scenario the expected winner is worked out by hand from the priority rules and the test waits for exactly that source to be offered, or asserts that a named source is never offered over a bounded window. The negative form matters as much as the positive one: a resolver that offers everything would satisfy every positive check.
- **Protocol rules.** For the register port the expected behaviour is the protocol itself, evaluated continuously rather than compared against a recorded trace.

The tests cover all band-urgency encodings but not every combination of source configuration, request timing and nesting history. The reference is a priority-selection oracle, not a cycle-accurate model of the full controller.

3 Verification environment

3.1 Overview

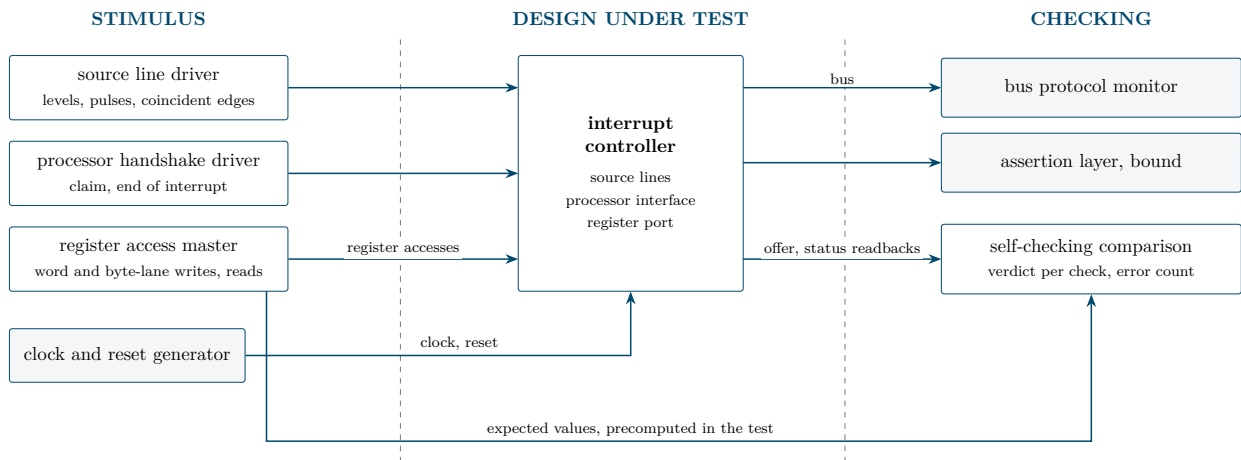


Figure 1: Block-level verification environment. Three drivers reach the controller and each of them exists because one class of behaviour is only reachable through it: the source line driver produces levels, single-cycle pulses and edges deliberately aligned with a claim; the handshake driver produces the claim and end-of-interrupt pulses a processor would produce, but at times a processor could not be made to choose; the register access master issues full-word and byte-lane writes and reads, and is the same driver in all six block-level tests, so every test exercises the slave in the same way. On the right, the bus protocol monitor and the bound assertion layer watch continuously and report violations of their own accord, while the self-checking comparison evaluates the register readbacks and the offer against values the test computed in advance. A run fails if any of the three reports a mismatch.

At system level the arrangement is different in one important way: nothing drives the processor interface. The processor decides for itself when to accept an offer and when to return, so the claim timing, the interval between the offer and the claim, and the nesting behaviour are produced by the real handshake rather than by a driver.

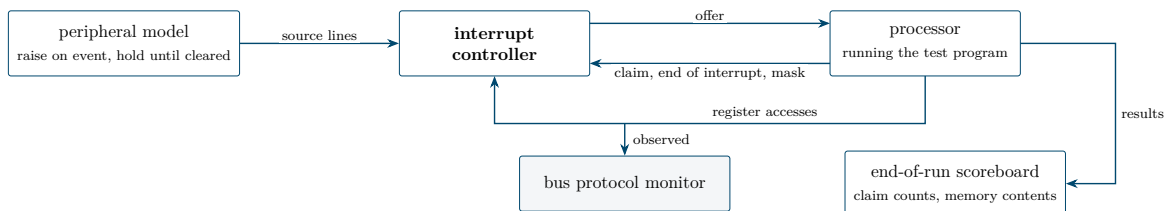


Figure 2: System-level arrangement. The environment plays the peripherals: it raises a source line on an event and holds it until the handler clears it, which is how a level-triggered peripheral behaves and is what makes the in-service suppression path reachable. The processor runs a program that configures the controller over the real register port, enables interrupts, and records what each handler saw; its claim and end-of-interrupt pulses and its own enable mask go back to the controller. The protocol monitor watches the register port throughout. At the end of the run the scoreboard compares the number of claims per source, the values the handlers left in memory and the processor’s final architectural state against precomputed values.

3.2 Control and drivers

3.2.1 Clock and reset generation

A single generator produces the clock and the asynchronous, active-low reset for every run. Reset release is the starting point of every test, which is what allows the reset test to read every register

before any other activity has taken place. The reset test additionally drives reset itself, so that it can apply it between clock edges from a configured, nested state.

3.2.2 Source line stimulus

At block level the source lines are driven directly, and the driver produces three distinct shapes because three distinct behaviours depend on them: a held level, a single-cycle pulse for an edge-triggered source, and a transition deliberately aligned with the same clock edge as a claim. The last of these is the only way to reach the arbitration between the edge latch being set and the same latch being consumed.

At system level the environment behaves as a peripheral would. It raises a source line when the program requests an event and lowers it when the controller signals that the source has been claimed, which models a device whose status register is cleared by its handler. Seven of the sixteen lines are driven this way; one further line is driven by a real block elsewhere in the system, so at least one source in the run is not under the environment's control at all.

3.2.3 Processor handshake stimulus

At block level the claim and end-of-interrupt pulses are driven by the test. Each is a single-cycle pulse, and the claim is deliberately issued one cycle after the offer has been observed, so that it lands on the source number the controller expects to be claimed. This is the same timing relationship the processor produces, and driving it explicitly is what allows the tests to place a claim at a moment the processor never would — immediately after a source has withdrawn, or while the depth is at its limit.

The processor's own enable mask is held at all ones in every block-level test except the scheduling test, which is the test that exists to exercise it.

3.2.4 Register access stimulus

One shared master task drives the register port in all six block-level tests. It performs full-word writes, byte-lane writes with an explicit strobe pattern, and reads, and it checks the response code of every access against the value the test expects — so an unexpected error response, or an unexpected success, is a failure even when the register value happens to be right.

At system level the register accesses are ordinary load and store instructions executed by the processor, which means the register interface is exercised through a real bus and a real decoder rather than through a task written for the purpose.

3.3 Monitors and checkers

3.3.1 Bus protocol monitor

A passive monitor is attached to the controller's register port in the system-level environment. It drives nothing and checks four families of rules:

- **Stability.** Once a channel asserts its valid signal it must hold it, and its payload, until the handshake completes.
- **Ordering and outstanding count.** A read response may only appear when a read is outstanding, a write response only after both write channels have been accepted, and a second request on either direction before its response breaks the one-outstanding contract.
- **Absence of undefined values.** No valid or ready signal, and no payload under an asserted valid signal, may be undefined after reset.

- **Response codes.** The exclusive-access response code does not exist in this protocol subset and must never appear.

The monitor also counts the reads and writes it has seen, and the run checks that both counts are non-zero. Without that, a port that was never accessed would pass every protocol rule trivially.

3.3.2 Assertion layer

An assertion layer is bound into the controller without modifying it. It contains the controller's own invariants and, separately, the same bus protocol rule set expressed as temporal properties on the register port.

The controller invariants are the ones that must hold in every cycle rather than at the end of a scenario:

- the request line and the source number presented to the processor are the registered result of one and the same resolution;
- an offered source is a pending request and is not already in service;
- an offered source is not one the processor's mask excludes;
- under nesting, the offered source's key is strictly greater than the key at the top of the nesting stack;
- the depth limit never exceeds the physical stack, and the depth never exceeds the limit;
- the claim and end-of-interrupt pulses never coincide;
- the depth changes only in response to a claim or an end of interrupt;
- no offer is presented while the depth is at its limit;
- an end of interrupt received with an empty stack leaves the depth unchanged.

ModelSim executes all functional tests and the ordinary protocol monitors. Verilator executes the same six PIC block benches and the CPU/SoC benches with the applicable bound assertions. No assertion failure was reported.

The feature test explicitly pulses EOI twice at depth zero and checks that nesting status, active vector and interrupt status remain zero. This reaches the empty-stack assertion's antecedent. Mutation testing removes the empty-stack guard and requires this test to fail. Two further PIC mutations remove CPU masking and reverse the source-index tie-break; the independent reference test detects both. A passing assertion with an unexercised antecedent is not credited as a tested scenario.

3.3.3 Self-checking comparison

Every check compares one observed value against one expected value. Mismatches are always printed; successful individual checks require `exttt+verbose`. A mismatch increments an error count, and a test ends by printing a pass banner if the count is zero and a failure banner otherwise. Tests do not stop at the first mismatch, so a single run reports every property it broke rather than only the first.

Two checking conventions are used in addition to the plain value comparison, and both exist because the plain comparison is too weak on its own:

- **Bounded negative checks.** Several properties are the absence of something — a masked source that must never be offered, an edge that must not re-fire, a preemption that must not be allowed. These are checked by watching a named source over a bounded window and failing if it appears at any point in it, not by sampling once.

- **Preconditions that make the check mean something.** A read-only register is only proven read-only if it held something other than zero at the time of the attempt. Every read-only register in the access-class test is therefore first driven to a non-zero value by real activity, and the write attempt is checked both for the error response and for the register still holding what the hardware put there.

3.4 Simulation environment setup

3.4.1 Structure

The environment is built in four layers, compiled in order into one library: the design; the shared verification models — clock and reset generation, the bus models, the passive protocol monitor, the shared register access driver and the shared comparison mechanism; the tests themselves; and, for the secondary flow only, the assertion layer and its binding.

Compiling every test into one library in a single pass, rather than per test, is what makes a name collision or an interface change between the design and a test a compile error rather than a silently skipped run.

3.4.2 Scripts

There are two entry points. The regression compiles once and then executes every test in turn, each in its own elaboration, printing a banner per run. The secondary flow compiles all fifteen CPU/PIC and sixteen SoC benches with a different front end and executes each with the applicable assertions enabled. The independent reference image is regenerated before running the PIC reference test.

Parameterised runs pass their configuration as an elaboration-time override and then read the value back out of the elaborated design and print it, so the log proves the configuration was actually applied rather than silently ignored.

3.4.3 Setup and configuration

The block-level tests take no configuration: each drives the controller from reset and programs it through its own register writes, so a run is reproducible without external settings. Several of them reset the controller between independent cases, so that one case cannot inherit state from the previous one.

The system-level runs differ only in the timing of the memory models: read latency, write latency, and the probability with which a model withholds its ready signal, with the random sequence seeded explicitly so a failing run can be repeated exactly.

4 Annex

4.1 Verification plan

Every item below is checked, and passed in the regression measured for this revision.

Table 5: Verification plan

ID	Title	Description	Status	Test, checker
Source handling				
VP-01	Level source request	A level line raises a request while it is high and withdraws it when it falls, with nothing latched.	Passing	Status test, both directions of the pending bit

ID	Title	Description	Status	Test, checker
VP-02	Edge source latching	A single-cycle pulse on an edge-configured source is captured and offered after the line has already fallen.	Passing	Feature test, offer observed
VP-03	Edge request consumed	A latched edge is cleared by the claim and does not re-fire after the handler returns.	Passing	Feature test, bounded negative check
VP-04	Edge in the claim cycle	A rising edge landing on the same edge as the claim survives it and is offered again after the return.	Passing	Scheduling test, offer observed after the return
VP-05	Per-source enable	A line held high produces no request, no offer and no pending indication while the source is disabled, and produces all three once it is enabled.	Passing	Reset test and status test, register readback
VP-06	Software trigger, keyed set	A full-word write carrying the key raises the request and the slot is offered.	Passing	Feature test, readback and offer observed
VP-07	Software trigger key guard	A write without the key, and a byte write that leaves the key lanes unstrobed, both leave the channel clear.	Passing	Feature test and scheduling test, bounded negative check
VP-08	Software trigger consumed	The channel clears itself when the slot is claimed.	Passing	Feature test, register readback
VP-09	Software trigger cleared	Writing zero withdraws a software request that has not been claimed.	Passing	Access-class test, register readback
Priority and scheduling				
VP-10	Inter-band priority	Of two sources requesting together, the one whose band carries the greater urgency is offered first.	Passing	Feature test, offer order
VP-11	Intra-band priority	Within one band the source with the greater intra-band priority is offered first.	Passing	Feature test, offer order
VP-12	Slot-number tie-break	Two sources with the same band and the same intra-band priority are ordered by slot number, lower first.	Passing	Feature test, offer order
VP-13	Band reordering	With the band urgencies reordered, the urgency and not the band number decides the outcome.	Passing	Scheduling test, offer order under a reordered configuration
VP-14	Processor mask in the resolution	A source the processor has masked is never offered, does not prevent a less urgent source behind it from being offered, and is offered on the first resolution after the mask lifts.	Passing	Scheduling test, bounded negative check plus offer observed

ID	Title	Description	Status	Test, checker
VP-15	Pending set contents	The pending set reports every pending source, including one held back by the processor's mask, and not only the offered one.	Passing	Scheduling test, port comparison
Nesting and preemption				
VP-16	Claim accounting	A claim raises the depth by one and the in-service source appears in the nesting status and active-source registers.	Passing	Feature test, register readback
VP-17	Preemption	A source more urgent than the level in service is offered while its handler runs, and its claim opens a further level rather than replacing the current one.	Passing	Feature test, depth and top-of-stack readback
VP-18	Unwinding	The inner end of interrupt restores the outer level as the top of stack, and the outer one returns the depth to zero.	Passing	Feature test, register readback
VP-19	In-service suppression	A level source still asserted while its own handler runs is not offered again until the end of interrupt.	Passing	Feature test and status test, offer and status readback
VP-20	Threshold is strict	A source that is not more urgent than the level in service is not offered at all.	Passing	Feature test, bounded negative check
VP-21	Live reconfiguration of an in-service source	Moving an in-service source to the most urgent band does not raise the threshold its handler runs under: a source that could preempt it before the change still does.	Passing	Feature test, depth readback after the preemption
VP-22	Depth limit	At the limit no preemption is offered whatever its priority, the overflow flag is set in both registers that expose it, and relaxing the limit lets the preemption through immediately.	Passing	Feature test, bounded negative check plus flag readback
VP-23	Depth limit clamping	A write of zero stores one, a write above the physical stack depth stores that depth, and a legal value is stored exactly.	Passing	Access-class test, readback of four written values
Deadline escalation				
VP-24	Escalation, jump mode	A source that waits longer than its deadline moves to the configured target band; the escalation flag, the effective band and the global event flag all change together.	Passing	Feature test and status test, status readback

ID	Title	Description	Status	Test, checker
VP-25	Escalation changes the offer	After escalating, a source that was previously outranked becomes the offered one, with no source line having moved.	Passing	Feature test, offer order
VP-26	Escalation preempts	An escalated source becomes urgent enough to preempt a source already in service.	Passing	Feature test, depth readback
VP-27	Bump mode under a reordered band order	With the bands reordered so that the numerically lower band is the less urgent one, a bump moves the source one step up the urgency order rather than one band number down.	Passing	Scheduling test, offer order plus escalation flag
VP-28	Repeated escalation	With repeats enabled the effective band walks up one step per elapsed deadline, and each observed transition moves exactly one band.	Passing	Feature test and status test, transitions counted
VP-29	Elapsed-time counter gating	The counter advances only while the source is pending and not in service, is cleared when the source is claimed and while it is idle, and stays at zero when no deadline is configured.	Passing	Status test, sampled across a full life cycle
VP-30	Escalation cleared at the end of the wait	The effective band returns to the configured band and the escalation flag clears once the source stops waiting.	Passing	Feature test and status test, status readback
Spurious claims				
VP-31	Spurious claim detection	A source that withdraws between the offer and the claim sets its per-source flag, its sticky log bit and the global event flag.	Passing	Feature test, status test and access-class test, register readback
VP-32	Legitimate claim not flagged	A source still asserted at the claim and cleared later inside its handler is not flagged, and the sticky log is untouched.	Passing	Status test, negative check
VP-33	Spurious claim accounted	The depth rises for a spurious claim as for any other, so claims and returns stay balanced.	Passing	Status test, depth readback
VP-34	Spurious indications cleared	The per-source flag clears when its level is released while the sticky log bit survives the return and is cleared only by software.	Passing	Status test, readback before and after the return
Register interface				

ID	Title	Description	Status	Test, checker
VP-35	Read-only enforcement	Every read-only register rejects a write with an error response and keeps the value the hardware put there. Each read-only register is driven to a non-zero value first, and all sixteen per-source instances are attempted individually.	Passing	Access-class test, response code plus readback
VP-36	Narrow writes to read-only registers	A byte write on each of the four lanes, and a write with no lanes enabled at all, are rejected exactly as a full-word write is, proving the rejection is decided from the address.	Passing	Access-class test, five attempts plus readback
VP-37	Unmapped offsets	Reads and writes to every unmapped offset in the window answer with an error response.	Passing	Feature test and access-class test, response code
VP-38	Reserved bits	After a write of all ones, every bit the map calls reserved reads zero, in each register that has one.	Passing	Access-class test, masked readback per register
VP-39	Write-only key field	The software-trigger key reads back as zero while the request bit reads its true state.	Passing	Access-class test, readback
VP-40	Write-one-to-clear behaviour	A written zero does not clear a set bit, a written one to a different bit does not clear it, and a written one to the bit itself does.	Passing	Access-class test, four-step sequence per register
VP-41	Byte-lane write to a writable register	A single-lane write to a writable register is accepted and updates that lane, which distinguishes an address-based rejection from a strobe-based one.	Passing	Access-class test, readback
VP-42	Bus protocol compliance	The register port obeys the handshake, ordering, outstanding-count and response-code rules of the protocol throughout the run, with traffic on both directions confirmed non-zero.	Passing	System test, passive protocol monitor
Reset				
VP-43	Reset values	Every mapped register reads its specified reset value on its first access, including all sixteen instances of each per-source register.	Passing	Reset test, register readback

ID	Title	Description	Status	Test, checker
VP-44	Interfaces during reset	While reset is asserted both bus valid signals are low, the response codes and read data are defined, and the request line and source number are inactive and defined.	Passing	Reset test, port comparison
VP-45	Internal state free of undefined values	The nesting stack arrays, which no register exposes directly, hold defined values after reset.	Passing	Reset test, direct inspection
VP-46	Asynchronous, dominant reset	Applied between clock edges from a fully configured state with interrupts in service, reset returns every register to its reset value and drops the request line without waiting for a clock edge.	Passing	Reset test, readback after re-release
VP-47	Disarmed out of reset	A source line held high across reset produces no request until software enables the source.	Passing	Reset test, bounded negative check
System integration				
VP-48	Integrated priority order	With two sources raised together, the more urgent one is served first, observed at the processor as the order in which the handlers are entered.	Passing	System test, claim timestamps
VP-49	Claim shape and timing	The claim is a single-cycle pulse, never asserted twice in succession, and never asserted while a data transaction is in flight.	Passing	System test, cycle-by-cycle invariants
VP-50	Claim count per source	Each source is claimed exactly the number of times the scenario demands over the whole run.	Passing	System test, per-source claim counters
VP-51	The two masks are independent	A source enabled in the controller but masked by the processor is never claimed, yet remains visible both in the pending set the processor reports and in its own status register.	Passing	System test, claim counter at zero plus register readback
VP-52	Offered source is enabled and not in service	Every source presented to the processor is one the controller had enabled when the offer was registered.	Passing	System test, cycle-by-cycle invariant
VP-53	In-service suppression at system level	A level source held high through its own handler is not offered again until the return, with the real handshake providing the timing.	Passing	System test, cycle-by-cycle invariant

ID	Title	Description	Status	Test, checker
VP-54	Register interface from software	The enable register, a per-source status register and the global status register read back their expected values when read by a program over real bus transactions, and the active-source register read from inside a handler names that handler's source.	Passing	System test, memory scoreboard
VP-55	Operation under bus timing variation	The whole system-level scenario holds under nominal timing, high fixed latency, and two seeds of randomised backpressure, which changes the interval between the offer and the claim.	Passing	System test, four configurations
VP-56	Preemption driven by a real processor	A handler re-enables the processor's global interrupt enable and is preempted from inside itself; the depth reaches two, each level is claimed once, and each return releases exactly its own level.	Passing	System test, depth and claim counters sampled every cycle
VP-57	Every slot reachable from software	Each of the sixteen slots in turn is raised through its software trigger channel and serviced; the source number the handler reads back matches the slot, including the eight slots no device is wired to.	Passing	System test, per-slot claim counters
VP-58	Escalation changes the service order	A source that misses its deadline while the processor is not yet accepting interrupts is serviced before the source that outranked it, and its neighbour without a deadline is left alone.	Passing	System test, order of handler entries and of claims
VP-59	Register port under response backpressure	A read or write response is held, with its payload, for as long as the master withholds its ready signal, and no further address is accepted meanwhile.	Passing	Backpressure test, 1, 3 and 10 cycle stalls
Continuous invariants				
VP-60	Offer is the registered resolution	The request line and source number presented are the registered result of one and the same resolution, in every cycle.	Passing	Assertion in applicable Verilator benches
VP-61	Offer eligibility holds continuously	Every offered source is pending, not in service, not masked by the processor, and under nesting strictly more urgent than the level in service.	Passing	Assertion in applicable Verilator benches

ID	Title	Description	Status	Test, checker
VP-62	Depth bounds and motion	The depth never exceeds the limit, the limit never exceeds the physical stack, the depth moves only on a claim or an end of interrupt, and no offer is made at the limit.	Passing	Assertion in applicable Verilator benches
VP-63	Claim and end of interrupt never coincide	The two pulses are never asserted in the same cycle in any run, which is the condition the depth accounting relies on.	Passing	Assertion in applicable Verilator benches

4.2 Measured results

The values below were measured on a regression run of the design revision this document describes.

Table 6: Regression result for the runs that exercise the controller

Run	Runs	Checks	Result
Feature test	1	145	pass
Priority reference	1	5 741	pass
Scheduling corner cases	1	26	pass
Reset	1	350	pass
Access classes and reserved bits	1	478	pass
Per-source status fields	1	424	pass
Processor system test, controller checks	4	108	pass
Source range, through the processor	1	36	pass
Preemption and nesting	1	16	pass
Deadline escalation	1	12	pass
Register port under backpressure	1	14	pass
Machine timer delivery, controller checks	1	4	pass
Interrupt masked at the peripheral	1	3	pass
Total	16	7 357	0 failures

The processor system test figure counts the 27 checks per run that bear on the controller, out of 97 in the nominal configuration and 95 in each of the other three. The last three rows likewise count only the controller’s share of runs whose subject is something else. The remaining checks in all of them are reported in the specifications for the blocks they belong to.

These runs are drawn from the CPU/PIC regression (21 runs, 8 473 labelled checks plus four independent ISA traces) and the SoC regression (25 runs, 444 labelled checks). Both complete without failures, compilation errors or ModelSim warnings. Labelled check counts include bus response checks and are not a measure of state-space coverage.

The Verilator flow executes every distinct functional bench in both groups with the applicable assertions. The independent priority reference and all three PIC mutation cases were executed; none was left as an unrun test.